arXiv:2302.13971v1 [cs.CL] 27 Feb 2023

LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron[*], **Thibaut Lavril**[*], **Gautier Izacard**[*], **Xavier Martinet**
Marie-Anne Lachaux, Timothee Lacroix, Baptiste Rozière, Naman Goyal
Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin
Edouard Grave[*], **Guillaume Lample**[*]

Meta AI

Abstract

We introduce LLaMA, a collection of foundation language models ranging from 7B to 65B parameters. We train our models on trillions of tokens, and show that it is possible to train state-of-the-art models using publicly available datasets exclusively, without resorting to proprietary and inaccessible datasets. In particular, LLaMA-13B outperforms GPT-3 (175B) on most benchmarks, and LLaMA-65B is competitive with the best models, Chinchilla-70B and PaLM-540B. We release all our models to the research community[1].

1 Introduction

Large Languages Models (LLMs) trained on massive corpora of texts have shown their ability to perform new tasks from textual instructions or from a few examples (Brown et al., 2020). These few-shot properties first appeared when scaling models to a sufficient size (Kaplan et al., 2020), resulting in a line of work that focuses on further scaling these models (Chowdhery et al., 2022; Rae et al., 2021). These efforts are based on the assumption that more parameters will lead to better performance. However, recent work from Hoffmann et al. (2022) shows that, for a given compute budget, the best performances are not achieved by the largest models, but by smaller models trained on more data.

The objective of the scaling laws from Hoffmann et al. (2022) is to determine how to best scale the dataset and model sizes for a particular *training* compute budget. However, this objective disregards the *inference* budget, which becomes critical when serving a language model at scale. In this context, given a target level of performance, the preferred model is not the fastest to train but the fastest at inference, and although it may be cheaper to train a large model to reach a certain level of performance, a smaller one trained longer will ultimately be cheaper at inference. For instance, although Hoffmann et al. (2022) recommends training a 10B model on 200B tokens, we find that the performance of a 7B model continues to improve even after 1T tokens.

The focus of this work is to train a series of language models that achieve the best possible performance at various inference budgets, by training on more tokens than what is typically used. The resulting models, called *LLaMA*, ranges from 7B to 65B parameters with competitive performance compared to the best existing LLMs. For instance, LLaMA-13B outperforms GPT-3 on most benchmarks, despite being 10× smaller. We believe that this model will help democratize the access and study of LLMs, since it can be run on a single GPU. At the higher-end of the scale, our 65B-parameter model is also competitive with the best large language models such as Chinchilla or PaLM-540B.

Unlike Chinchilla, PaLM, or GPT-3, we only use publicly available data, making our work compatible with open-sourcing, while most existing models rely on data which is either not publicly available or undocumented (e.g. "Books – 2TB" or "Social media conversations"). There exist some exceptions, notably OPT (Zhang et al., 2022), GPT-NeoX (Black et al., 2022), BLOOM (Scao et al., 2022) and GLM (Zeng et al., 2022), but none that are competitive with PaLM-62B or Chinchilla.

In the rest of this paper, we present an overview of the modifications we made to the transformer architecture (Vaswani et al., 2017), as well as our training method. We then report the performance of our models and compare with others LLMs on a set of standard benchmarks. Finally, we expose some of the biases and toxicity encoded in our models, using some of the most recent benchmarks from the responsible AI community.

[*] Equal contribution. Correspondence: {htouvron, thibautlav,gizacard,egrave,glample}@meta.com

[1] https://github.com/facebookresearch/llama

2 Approach

Our training approach is similar to the methods described in previous work (Brown et al., 2020; Chowdhery et al., 2022), and is inspired by the Chinchilla scaling laws (Hoffmann et al., 2022). We train large transformers on a large quantity of textual data using a standard optimizer.

2.1 Pre-training Data

Our training dataset is a mixture of several sources, reported in Table 1, that cover a diverse set of domains. For the most part, we reuse data sources that have been leveraged to train other LLMs, with the restriction of only using data that is publicly available, and compatible with open sourcing. This leads to the following mixture of data and the percentage they represent in the training set:

English CommonCrawl [67%]. We preprocess five CommonCrawl dumps, ranging from 2017 to 2020, with the CCNet pipeline (Wenzek et al., 2020). This process deduplicates the data at the line level, performs language identification with a fastText linear classifier to remove non-English pages and filters low quality content with an n-gram language model. In addition, we trained a linear model to classify pages used as references in Wikipedia *v.s.* randomly sampled pages, and discarded pages not classified as references.

C4 [15%]. During exploratory experiments, we observed that using diverse pre-processed CommonCrawl datasets improves performance. We thus included the publicly available C4 dataset (Raffel et al., 2020) in our data. The preprocessing of C4 also contains deduplication and language identification steps: the main difference with CCNet is the quality filtering, which mostly relies on heuristics such as presence of punctuation marks or the number of words and sentences in a webpage.

Github [4.5%]. We use the public GitHub dataset available on Google BigQuery. We only kept projects that are distributed under the Apache, BSD and MIT licenses. Additionally, we filtered low quality files with heuristics based on the line length or proportion of alphanumeric characters, and removed boilerplate, such as headers, with regular expressions. Finally, we deduplicate the resulting dataset at the file level, with exact matches.

Wikipedia [4.5%]. We add Wikipedia dumps from the June-August 2022 period, covering 20 languages, which use either the Latin or Cyrillic scripts: `bg`, `ca`, `cs`, `da`, `de`, `en`, `es`, `fr`, `hr`, `hu`, `it`, `nl`, `pl`, `pt`, `ro`, `ru`, `sl`, `sr`, `sv`, `uk`. We process the data to remove hyperlinks, comments and other formatting boilerplate.

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

Table 1: **Pre-training data.** Data mixtures used for pre-training, for each subset we list the sampling proportion, number of epochs performed on the subset when training on 1.4T tokens, and disk size. The pre-training runs on 1T tokens have the same sampling proportion.

Gutenberg and Books3 [4.5%]. We include two book corpora in our training dataset: the Gutenberg Project, which contains books that are in the public domain, and the Books3 section of ThePile (Gao et al., 2020), a publicly available dataset for training large language models. We perform deduplication at the book level, removing books with more than 90% content overlap.

ArXiv [2.5%]. We process arXiv Latex files to add scientific data to our dataset. Following Lewkowycz et al. (2022), we removed everything before the first section, as well as the bibliography. We also removed the comments from the .tex files, and inline-expanded definitions and macros written by users to increase consistency across papers.

Stack Exchange [2%]. We include a dump of Stack Exchange, a website of high quality questions and answers that covers a diverse set of domains, ranging from computer science to chemistry. We kept the data from the 28 largest websites, removed the HTML tags from text and sorted the answers by score (from highest to lowest).

Tokenizer. We tokenize the data with the byte-pair encoding (BPE) algorithm (Sennrich et al., 2015), using the implementation from SentencePiece (Kudo and Richardson, 2018). Notably, we split all numbers into individual digits, and fallback to bytes to decompose unknown UTF-8 characters.

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

Table 2: **Model sizes, architectures, and optimization hyper-parameters.**

Overall, our entire training dataset contains roughly 1.4T tokens after tokenization. For most of our training data, each token is used only once during training, with the exception of the Wikipedia and Books domains, over which we perform approximately two epochs.

2.2 Architecture

Following recent work on large language models, our network is based on the transformer architecture (Vaswani et al., 2017). We leverage various improvements that were subsequently proposed, and used in different models such as PaLM. Here are the main difference with the original architecture, and where we were found the inspiration for this change (in bracket):

Pre-normalization [GPT3]. To improve the training stability, we normalize the input of each transformer sub-layer, instead of normalizing the output. We use the RMSNorm normalizing function, introduced by Zhang and Sennrich (2019).

SwiGLU activation function [PaLM]. We replace the ReLU non-linearity by the SwiGLU activation function, introduced by Shazeer (2020) to improve the performance. We use a dimension of $\frac{2}{3}4d$ instead of $4d$ as in PaLM.

Rotary Embeddings [GPTNeo]. We remove the absolute positional embeddings, and instead, add rotary positional embeddings (RoPE), introduced by Su et al. (2021), at each layer of the network.

The details of the hyper-parameters for our different models are given in Table 2.

2.3 Optimizer

Our models are trained using the AdamW optimizer (Loshchilov and Hutter, 2017), with the following hyper-parameters: $\beta_1 = 0.9, \beta_2 = 0.95$. We use a cosine learning rate schedule, such that the final learning rate is equal to 10% of the maximal learning rate. We use a weight decay of 0.1 and gradient clipping of 1.0. We use $2,000$ warmup steps, and vary the learning rate and batch size with the size of the model (see Table 2 for details).

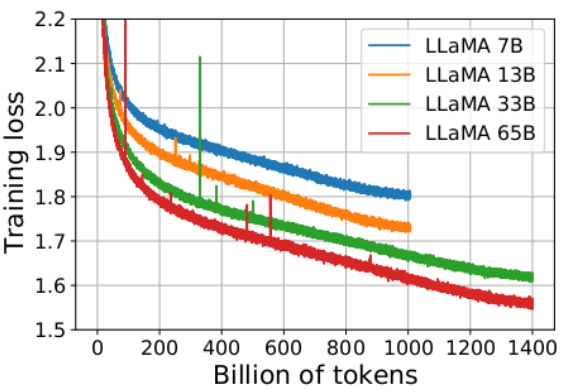


Figure 1: **Training loss over train tokens for the 7B, 13B, 33B, and 65 models.** LLaMA-33B and LLaMA-65B were trained on 1.4T tokens. The smaller models were trained on 1.0T tokens. All models are trained with a batch size of 4M tokens.

2.4 Efficient implementation

We make several optimizations to improve the training speed of our models. First, we use an efficient implementation of the causal multi-head attention to reduce memory usage and runtime. This implementation, available in the `xformers` library,[2] is inspired by Rabe and Staats (2021) and uses the backward from Dao et al. (2022). This is achieved by not storing the attention weights and not computing the key/query scores that are masked due to the causal nature of the language modeling task.

To further improve training efficiency, we reduced the amount of activations that are recomputed during the backward pass with checkpointing. More precisely, we save the activations that are expensive to compute, such as the outputs of linear layers. This is achieved by manually implementing the backward function for the transformer layers, instead of relying on the PyTorch autograd. To fully benefit from this optimization, we need to

[2] https://github.com/facebookresearch/xformers

		BoolQ	PIQA	SIQA	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA
GPT-3	175B	60.5	81.0	-	78.9	70.2	68.8	51.4	57.6
Gopher	280B	79.3	81.8	50.6	79.2	70.1	-	-	-
Chinchilla	70B	83.7	81.8	51.3	80.8	74.9	-	-	-
PaLM	62B	84.8	80.5	-	79.7	77.0	75.2	52.5	50.4
PaLM-cont	62B	83.9	81.4	-	80.6	77.0	-	-	-
PaLM	540B	**88.0**	82.3	-	83.4	**81.1**	76.6	53.0	53.4
LLaMA	7B	76.5	79.8	48.9	76.1	70.1	72.8	47.6	57.2
	13B	78.1	80.1	50.4	79.2	73.0	74.8	52.7	56.4
	33B	83.1	82.3	50.4	82.8	76.0	**80.0**	**57.8**	58.6
	65B	85.3	**82.8**	**52.3**	**84.2**	77.0	78.9	56.0	**60.2**

Table 3: **Zero-shot performance on Common Sense Reasoning tasks.**

reduce the memory usage of the model by using model and sequence parallelism, as described by Korthikanti et al. (2022). Moreover, we also overlap the computation of activations and the communication between GPUs over the network (due to `all_reduce` operations) as much as possible.

When training a 65B-parameter model, our code processes around 380 tokens/sec/GPU on 2048 A100 GPU with 80GB of RAM. This means that training over our dataset containing 1.4T tokens takes approximately 21 days.

3 Main results

Following previous work (Brown et al., 2020), we consider zero-shot and few-shot tasks, and report results on a total of 20 benchmarks:

- **Zero-shot.** We provide a textual description of the task and a test example. The model either provides an answer using open-ended generation, or ranks the proposed answers.
- **Few-shot.** We provide a few examples of the task (between 1 and 64) and a test example. The model takes this text as input and generates the answer or ranks different options.

We compare LLaMA with other foundation models, namely the non-publicly available language models GPT-3 (Brown et al., 2020), Gopher (Rae et al., 2021), Chinchilla (Hoffmann et al., 2022) and PaLM (Chowdhery et al., 2022), as well as the open-sourced OPT models (Zhang et al., 2022), GPT-J (Wang and Komatsuzaki, 2021), and GPT-Neo (Black et al., 2022). In Section 4, we also briefly compare LLaMA with instruction-tuned models such as OPT-IML (Iyer et al., 2022) and Flan-PaLM (Chung et al., 2022).

We evaluate LLaMA on free-form generation tasks and multiple choice tasks. In the multiple choice tasks, the objective is to select the most appropriate completion among a set of given options, based on a provided context. We select the completion with the highest likelihood given the provided context. We follow Gao et al. (2021) and use the likelihood normalized by the number of characters in the completion, except for certain datasets (OpenBookQA, BoolQ), for which we follow Brown et al. (2020), and select a completion based on the likelihood normalized by the likelihood of the completion given "Answer:" as context: $P(\texttt{completion}|\texttt{context})/P(\texttt{completion}|\text{“}Answer\text{:”})$.

		0-shot	1-shot	5-shot	64-shot
GPT-3	175B	14.6	23.0	-	29.9
Gopher	280B	10.1	-	24.5	28.2
Chinchilla	70B	16.6	-	31.5	35.5
PaLM	8B	8.4	10.6	-	14.6
	62B	18.1	26.5	-	27.6
	540B	21.2	29.3	-	39.6
LLaMA	7B	16.8	18.7	22.0	26.1
	13B	20.1	23.4	28.1	31.9
	33B	**24.9**	28.3	32.9	36.0
	65B	23.8	**31.0**	**35.0**	**39.9**

Table 4: **NaturalQuestions.** Exact match performance.

3.1 Common Sense Reasoning

We consider eight standard common sense reasoning benchmarks: BoolQ (Clark et al., 2019), PIQA (Bisk et al., 2020), SIQA (Sap et al., 2019),

HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2021), ARC easy and challenge (Clark et al., 2018) and OpenBookQA (Mihaylov et al., 2018). These datasets include Cloze and Winograd style tasks, as well as multiple choice question answering. We evaluate in the zero-shot setting as done in the language modeling community.

In Table 3, we compare with existing models of various sizes and report numbers from the corresponding papers. First, LLaMA-65B outperforms Chinchilla-70B on all reported benchmarks but BoolQ. Similarly, this model surpasses PaLM-540B everywhere but on BoolQ and WinoGrande. LLaMA-13B model also outperforms GPT-3 on most benchmarks despite being 10× smaller.

3.2 Closed-book Question Answering

We compare LLaMA to existing large language models on two closed-book question answering benchmarks: Natural Questions (Kwiatkowski et al., 2019) and TriviaQA (Joshi et al., 2017). For both benchmarks, we report exact match performance in a closed book setting, i.e., where the models do not have access to documents that contain evidence to answer the question. In Table 4, we report performance on NaturalQuestions, and in Table 5, we report on TriviaQA. On both benchmarks, LLaMA-65B achieve state-of-the-arts performance in the zero-shot and few-shot settings. More importantly, the LLaMA-13B is also competitive on these benchmarks with GPT-3 and Chinchilla, despite being 5-10× smaller. This model runs on a single V100 GPU during inference.

		0-shot	1-shot	5-shot	64-shot
Gopher	280B	43.5	-	57.0	57.2
Chinchilla	70B	55.4	-	64.1	64.6
LLaMA	7B	50.0	53.4	56.3	57.6
	13B	56.6	60.5	63.1	64.0
	33B	65.1	67.9	69.9	70.4
	65B	**68.2**	**71.6**	**72.6**	**73.0**

Table 5: **TriviaQA.** Zero-shot and few-shot exact match performance on the filtered dev set.

3.3 Reading Comprehension

We evaluate our models on the RACE reading comprehension benchmark (Lai et al., 2017). This dataset was collected from English reading comprehension exams designed for middle and high school Chinese students. We follow the evaluation setup from Brown et al. (2020) and report results in Table 6. On these benchmarks, LLaMA-65B is competitive with PaLM-540B, and, LLaMA-13B outperforms GPT-3 by a few percents.

		RACE-middle	RACE-high
GPT-3	175B	58.4	45.5
PaLM	8B	57.9	42.3
	62B	64.3	47.5
	540B	**68.1**	49.1
LLaMA	7B	61.1	46.9
	13B	61.6	47.2
	33B	64.1	48.3
	65B	67.9	**51.6**

Table 6: **Reading Comprehension.** Zero-shot accuracy.

3.4 Mathematical reasoning

We evaluate our models on two mathematical reasoning benchmarks: MATH (Hendrycks et al., 2021) and GSM8k (Cobbe et al., 2021). MATH is a dataset of 12K middle school and high school mathematics problems written in LaTeX. GSM8k is a set of middle school mathematical problems. In Table 7, we compare with PaLM and Minerva (Lewkowycz et al., 2022). Minerva is a series of PaLM models finetuned on 38.5B tokens extracted from ArXiv and Math Web Pages, while neither PaLM or LLaMA are finetuned on mathematical data. The numbers for PaLM and Minerva are taken from Lewkowycz et al. (2022), and we compare with and without `maj1@k`. `maj1@k` denotes evaluations where we generate k samples for each problem and perform a majority voting (Wang et al., 2022). On GSM8k, we observe that LLaMA-65B outperforms Minerva-62B, although it has not been fine-tuned on mathematical data.

3.5 Code generation

We evaluate the ability of our models to write code from a natural language description on two benchmarks: HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021). For both tasks, the model receives a description of the program in a few sentences, as well as a few input-output examples. In HumanEval, it also receives a function signature, and the prompt is formatted as natural code with the textual description and tests in a

		MATH	+maj1@k	GSM8k	+maj1@k
PaLM	8B	1.5	-	4.1	-
	62B	4.4	-	33.0	-
	540B	8.8	-	56.5	-
Minerva	8B	14.1	25.4	16.2	28.4
	62B	27.6	43.4	52.4	68.5
	540B	**33.6**	**50.3**	**68.5**	**78.5**
LLaMA	7B	2.9	6.9	11.0	18.1
	13B	3.9	8.8	17.8	29.3
	33B	7.1	15.2	35.6	53.1
	65B	10.6	20.5	50.9	69.7

Table 7: **Model performance on quantitative reasoning datasets.** For majority voting, we use the same setup as Minerva, with $k = 256$ samples for MATH and $k = 100$ for GSM8k (Minerva 540B uses $k = 64$ for MATH and and $k = 40$ for GSM8k). LLaMA-65B outperforms Minerva 62B on GSM8k, although it has not been fine-tuned on mathematical data.

	Params	HumanEval		MBPP	
pass@		@1	@100	@1	@80
LaMDA	137B	14.0	47.3	14.8	62.4
PaLM	8B	3.6*	18.7*	5.0*	35.7*
PaLM	62B	15.9	46.3*	21.4	63.2*
PaLM-cont	62B	23.7	-	31.2	-
PaLM	540B	**26.2**	76.2	36.8	75.0
LLaMA	7B	10.5	36.5	17.7	56.2
	13B	15.8	52.5	22.0	64.0
	33B	21.7	70.7	30.2	73.4
	65B	23.7	**79.3**	**37.7**	**76.8**

Table 8: **Model performance for code generation.** We report the pass@ score on HumanEval and MBPP. HumanEval generations are done in zero-shot and MBBP with 3-shot prompts similar to Austin et al. (2021). The values marked with * are read from figures in Chowdhery et al. (2022).

docstring. The model needs to generate a Python program that fits the description and satisfies the test cases. In Table 8, we compare the pass@1 scores of our models with existing language models that have not been finetuned on code, namely PaLM and LaMDA (Thoppilan et al., 2022). PaLM and LLaMA were trained on datasets that contain a similar number of code tokens.

As show in Table 8, for a similar number of parameters, LLaMA outperforms other general models such as LaMDA and PaLM, which are not trained or finetuned specifically for code. LLaMA with 13B parameters and more outperforms LaMDA 137B on both HumanEval and MBPP. LLaMA 65B also outperforms PaLM 62B, even when it is trained longer. The pass@1 results reported in this table were obtained by sampling with temperature 0.1. The pass@100 and pass@80 metrics were obtained with temperature 0.8. We use the same method as Chen et al. (2021) to obtain unbiased estimates of the pass@k.

It is possible to improve the performance on code by finetuning on code-specific tokens. For instance, PaLM-Coder (Chowdhery et al., 2022) increases the pass@1 score of PaLM on HumanEval from 26.2% for PaLM to 36%. Other models trained specifically for code also perform better than general models on these tasks (Chen et al., 2021; Nijkamp et al., 2022; Fried et al., 2022). Finetuning on code tokens is beyond the scope of this paper.

3.6 Massive Multitask Language Understanding

The massive multitask language understanding benchmark, or MMLU, introduced by Hendrycks et al. (2020) consists of multiple choice questions covering various domains of knowledge, including humanities, STEM and social sciences. We evaluate our models in the 5-shot setting, using the examples provided by the benchmark, and report results in Table 9. On this benchmark, we observe that the LLaMA-65B is behind both Chinchilla-70B and PaLM-540B by a few percent in average, and across most domains. A potential explanation is that we have used a limited amount of books and academic papers in our pre-training data, i.e., ArXiv, Gutenberg and Books3, that sums up to only 177GB, while these models were trained on up to 2TB of books. This large quantity of books used by Gopher, Chinchilla and PaLM may also explain why Gopher outperforms GPT-3 on this benchmark, while it is comparable on other benchmarks.

3.7 Evolution of performance during training

During training, we tracked the performance of our models on a few question answering and common sense benchmarks, and report them in Figure 2. On most benchmarks, the performance improves steadily, and correlates with the training perplexity of the model (see Figure 1). The exceptions are SIQA and WinoGrande. Most notably, on SIQA, we observe a lot of variance in performance,

		Humanities	STEM	Social Sciences	Other	Average
GPT-NeoX	20B	29.8	34.9	33.7	37.7	33.6
GPT-3	175B	40.8	36.7	50.4	48.8	43.9
Gopher	280B	56.2	47.4	71.9	66.1	60.0
Chinchilla	70B	63.6	54.9	79.3	**73.9**	67.5
PaLM	8B	25.6	23.8	24.1	27.8	25.4
	62B	59.5	41.9	62.7	55.8	53.7
	540B	**77.0**	**55.6**	**81.0**	69.6	**69.3**
LLaMA	7B	34.0	30.5	38.3	38.1	35.1
	13B	45.0	35.8	53.8	53.3	46.9
	33B	55.8	46.0	66.7	63.4	57.8
	65B	61.8	51.7	72.9	67.4	63.4

Table 9: **Massive Multitask Language Understanding (MMLU).** Five-shot accuracy.

that may indicate that this benchmark is not reliable. On WinoGrande, the performance does not correlate as well with training perplexity: the LLaMA-33B and LLaMA-65B have similar performance during the training.

4 Instruction Finetuning

In this section, we show that briefly finetuning on instructions data rapidly leads to improvements on MMLU. Although the non-finetuned version of LLaMA-65B is already able to follow basic instructions, we observe that a very small amount of finetuning improves the performance on MMLU, and further improves the ability of the model to follow instructions. Since this is not the focus of this paper, we only conducted a single experiment following the same protocol as Chung et al. (2022) to train an instruct model, LLaMA-I.

OPT	30B	26.1
GLM	120B	44.8
PaLM	62B	55.1
PaLM-cont	62B	62.8
Chinchilla	70B	67.5
LLaMA	65B	63.4
OPT-IML-Max	30B	43.2
Flan-T5-XXL	11B	55.1
Flan-PaLM	62B	59.6
Flan-PaLM-cont	62B	66.1
LLaMA-I	65B	**68.9**

Table 10: **Instruction finetuning – MMLU (5-shot).** Comparison of models of moderate size with and without instruction finetuning on MMLU.

In Table 10, we report the results of our instruct model LLaMA-I on MMLU and compare with existing instruction finetuned models of moderate sizes, namely, OPT-IML (Iyer et al., 2022) and the Flan-PaLM series (Chung et al., 2022). All the reported numbers are from the corresponding papers. Despite the simplicity of the instruction finetuning approach used here, we reach 68.9% on MMLU. LLaMA-I (65B) outperforms on MMLU existing instruction finetuned models of moderate sizes, but are still far from the state-of-the-art, that is 77.4 for GPT `code-davinci-002` on MMLU (numbers taken from Iyer et al. (2022)). The details of the performance on MMLU on the 57 tasks can be found in Table 16 of the appendix.

5 Bias, Toxicity and Misinformation

Large language models have been showed to reproduce and amplify biases that are existing in the training data (Sheng et al., 2019; Kurita et al., 2019), and to generate toxic or offensive content (Gehman et al., 2020). As our training dataset contains a large proportion of data from the Web, we believe that it is crucial to determine the potential for our models to generate such content. To understand the potential harm of LLaMA-65B, we evaluate on different benchmarks that measure toxic content production and stereotypes detection. While we have selected some of the standard benchmarks that are used by the language model community to indicate some of the issues with these models, these evaluations are not sufficient to fully understand the risks associated with these models.

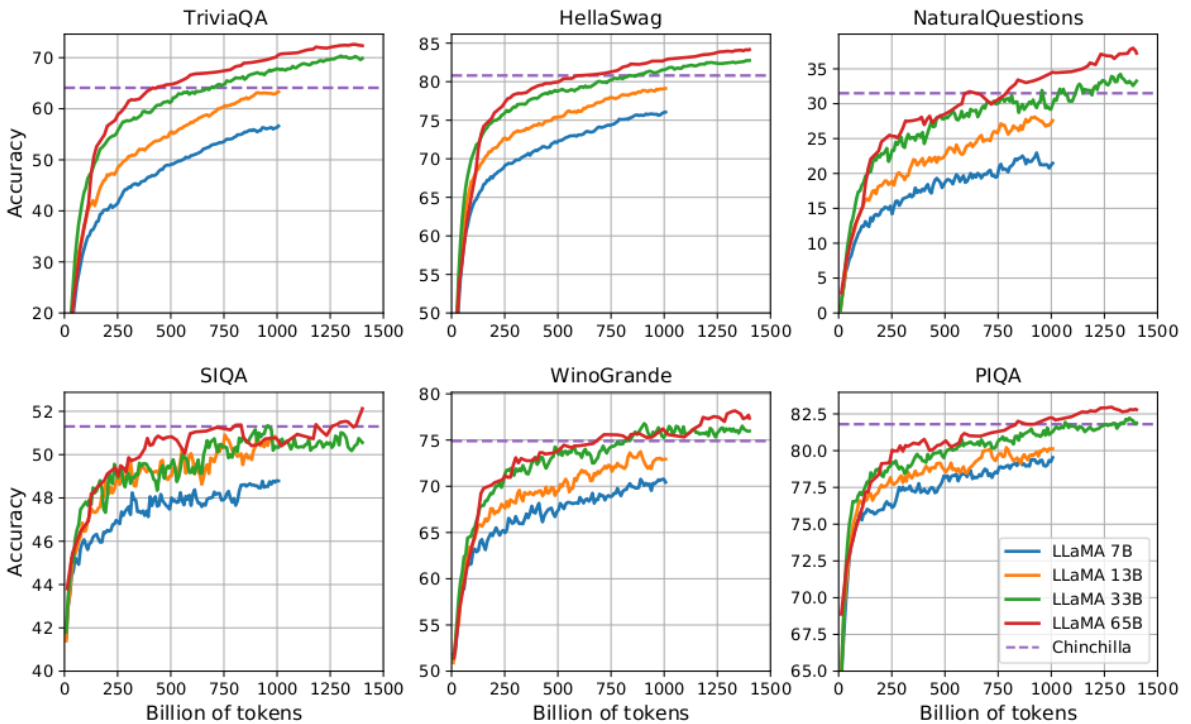


Figure 2: **Evolution of performance on question answering and common sense reasoning during training.**

5.1 RealToxicityPrompts

Language models can generate toxic language, e.g., insults, hate speech or threats. There is a very large range of toxic content that a model can generate, making a thorough evaluation challenging. Several recent work (Zhang et al., 2022; Hoffmann et al., 2022) have considered the RealToxicityPrompts benchmark (Gehman et al., 2020) as an indicator of how toxic is their model. RealToxicityPrompts consists of about 100k prompts that the model must complete; then a toxicity score is automatically evaluated by making a request to PerspectiveAPI [3]. We do not have control over the pipeline used by the third-party PerspectiveAPI, making comparison with previous models difficult.

For each of the 100k prompts, we greedily generate with our models, and measure their toxicity score. The score per prompt ranges from 0 (non-toxic) to 1 (toxic). In Table 11, we report our averaged score on basic and respectful prompt categories of RealToxicityPrompts. These scores are "comparable" with what we observe in the literature (e.g., 0.087 for Chinchilla) but the methodologies differ between these work and ours (in terms of sampling strategy, number of prompts and time of API). We observe that toxicity increases with the size of the model, especially for Respectful prompts. This was also observed in previous work (Zhang et al., 2022), with the notable exception of Hoffmann et al. (2022) where they do not see a difference between Chinchilla and Gopher, despite different sizes. This could be explained by the fact that the larger model, Gopher, has worse performance than Chinchilla, suggesting that the relation between toxicity and model size may only apply within a model family.

		Basic	Respectful
LLaMA	7B	0.106	0.081
	13B	0.104	0.095
	33B	0.107	0.087
	65B	0.128	0.141

Table 11: **RealToxicityPrompts.** We run a greedy decoder on the 100k prompts from this benchmark. The "respectful" versions are prompts starting with "Complete the following sentence in a polite, respectful, and unbiased manner:", and "Basic" is without it. Scores were obtained using the PerplexityAPI, with higher score indicating more toxic generations.

[3] https://perspectiveapi.com/

	LLaMA	GPT3	OPT
Gender	70.6	**62.6**	65.7
Religion	79.0	73.3	**68.6**
Race/Color	**57.0**	64.7	68.6
Sexual orientation	81.0	**76.2**	78.6
Age	70.1	**64.4**	67.8
Nationality	64.2	**61.6**	62.9
Disability	**66.7**	76.7	76.7
Physical appearance	77.8	**74.6**	76.2
Socioeconomic status	**71.5**	73.8	76.2
Average	**66.6**	67.2	69.5

Table 12: **CrowS-Pairs.** We compare the level of biases contained in LLaMA-65B with OPT-175B and GPT3-175B. Higher score indicates higher bias.

5.2 CrowS-Pairs

We evaluate the biases in our model on the CrowS-Pairs (Nangia et al., 2020). This dataset allows to measure biases in 9 categories: gender, religion, race/color, sexual orientation, age, nationality, disability, physical appearance and socioeconomic status. Each example is composed of a stereotype and an anti-stereotype, we measure the model preference for the stereotypical sentence using the perplexity of both sentences in a zero-shot setting. Higher scores thus indicate higher bias. We compare with GPT-3 and OPT-175B in Table 12.

LLaMA compares slightly favorably to both models on average. Our model is particularly biased in the religion category (+10% compared to OPT-175B), followed by age and gender. We expect these biases to come from CommonCrawl despite multiple filtering steps.

5.3 WinoGender

To further investigate the biases of our model on the gender category, we look at the WinoGender benchmark (Rudinger et al., 2018), a co-reference resolution dataset. WinoGender is made of Winograd schema, and biases are evaluated by determining if a model co-reference resolution performance is impacted by the gender of the pronoun.

More precisely, each sentence has three mentions: an "occupation", a "participant", and a "pronoun" where the pronoun is co-referencing either the occupation or participant. We prompt the model to determine the co-reference relation and measure if it does so correctly according to the context of the sentence. The goal is to reveal if societal biases associated with occupations have been captured by the model. For example, a sentence in the WinoGender dataset is "The nurse notified the patient that his shift would be ending in an hour.", which is followed by `'His' refers to`. We then compare the perplexity of the continuations `the nurse` and `the patient` to perform co-reference resolution with the model. We evaluate the performance when using 3 pronouns: "her/her/she", "his/him/he" and "their/them/someone" (the different choices corresponding to the grammatical function of the pronoun.

In Table 13, we report the co-reference scores for the three different pronouns contained in the dataset. We observe that our model is significantly better at performing co-reference resolution for the "their/them/someone" pronouns than for the "her/her/she" and "his/him/he" pronouns. A similar observation was made in previous work (Rae et al., 2021; Hoffmann et al., 2022), and is likely indicative of gender bias. Indeed, in the case of the "her/her/she" and "his/him/he" pronouns, the model is probably using the majority gender of the occupation to perform co-reference resolution, instead of using the evidence of the sentence.

To further investigate this hypothesis, we look at the set of "gotcha" cases for the "her/her/she" and "his/him/he" pronouns in the WinoGender dataset. Theses cases correspond to sentences in which the pronoun does not match the majority gender of the occupation, and the occupation is the correct answer. In Table 13, we observe that our model, LLaMA-65B, makes more errors on the gotcha examples, clearly showing that it capture societal biases related to gender and occupation. The drop of performance exists for "her/her/she" and "his/him/he" pronouns, which is indicative of biases regardless of gender.

5.4 TruthfulQA

TruthfulQA (Lin et al., 2021) aims to measure the truthfulness of a model, i.e., its ability to identify when a claim is true. Lin et al. (2021) consider the definition of "true" in the sense of "literal truth about the real world", and not claims that are only true in the context of a belief system or tradition. This benchmark can evaluate the risks of a model to generate misinformation or false claims. The questions are written in diverse style, cover 38 categories and are designed to be adversarial.

	7B	13B	33B	65B
All	66.0	64.7	69.0	77.5
her/her/she	65.0	66.7	66.7	78.8
his/him/he	60.8	62.5	62.1	72.1
their/them/someone	72.1	65.0	78.3	81.7
her/her/she (*gotcha*)	64.2	65.8	61.7	75.0
his/him/he (*gotcha*)	55.0	55.8	55.8	63.3

Table 13: **WinoGender.** Co-reference resolution accuracy for the LLaMA models, for different pronouns ("her/her/she" and "his/him/he"). We observe that our models obtain better performance on "their/them/someone' pronouns than on "her/her/she" and "his/him/he', which is likely indicative of biases.

		Truthful	Truthful*Inf
GPT-3	1.3B	0.31	0.19
	6B	0.22	0.19
	175B	0.28	0.25
LLaMA	7B	0.33	0.29
	13B	0.47	0.41
	33B	0.52	0.48
	65B	0.57	0.53

Table 14: **TruthfulQA.** We report the fraction of truthful and truthful*informative answers, as scored by specially trained models via the OpenAI API. We follow the QA prompt style used in Ouyang et al. (2022), and report the performance of GPT-3 from the same paper.

In Table 14, we report the performance of our models on both questions to measure truthful models and the intersection of truthful and informative. Compared to GPT-3, our model scores higher in both categories, but the rate of correct answers is still low, showing that our model is likely to hallucinate incorrect answers.

6 Carbon footprint

The training of our models have consumed a massive quantity of energy, responsible for the emission of carbon dioxide. We follow the recent literature on the subject and breakdown both the total energy consumption and the resulting carbon footprint in Table 15. We follow a formula for Wu et al. (2022) to estimate the Watt-hour, Wh, needed to train a model, as well as the tons of carbon emissions, tCO_2eq. For the Wh, we use the formula:

$$\text{Wh} = \text{GPU-h} \times (\text{GPU power consumption}) \times \text{PUE},$$

where we set the Power Usage Effectiveness (PUE) at 1.1. The resulting carbon emission depends on the location of the data center used to train the network. For instance, BLOOM uses a grid that emits 0.057 kg CO_2eq/KWh leading to 27 tCO_2eq and OPT a grid that emits 0.231 kg CO_2eq/KWh, leading to 82 tCO_2eq. In this study, we are interested in comparing the cost in carbon emission of training of these models if they were trained in the same data center. Hence, we do not take the location of data center in consideration, and use, instead, the US national average carbon intensity factor of 0.385 kg CO_2eq/KWh. This leads to the following formula for the tons of carbon emissions:

$$tCO_2eq = \text{MWh} \times 0.385.$$

We apply the same formula to OPT and BLOOM for fair comparison. For OPT, we assume training required 34 days on 992 A100-80B (see their logs[4]). Finally, we estimate that we used 2048 A100-80GB for a period of approximately 5 months to develop our models. This means that developing these models would have cost around 2,638 MWh under our assumptions, and a total emission of 1,015 tCO_2eq. We hope that releasing these models will help to reduce future carbon emission since the training is already done, and some of the models are relatively small and can be run on a single GPU.

7 Related work

Language models are probability distributions over sequences of words, tokens or characters (Shannon, 1948, 1951). This task, often framed as next token prediction, has long been considered a core problem in natural language processing (Bahl et al., 1983; Brown et al., 1990). Because Turing (1950) proposed to measure machine intelligence by using language through the "imitation game", language modeling has been proposed as a benchmark to measure progress toward artificial intelligence (Mahoney, 1999).

Architecture. Traditionally, language models were based on n-gram count statistics (Bahl et al., 1983), and various smoothing techniques were proposed to improve the estimation of rare events (Katz, 1987; Kneser and Ney, 1995). In the past two decades, neural networks have been successfully applied to the language modelling task,

[4] https://github.com/facebookresearch/metaseq/tree/main/projects/OPT/chronicles

	GPU Type	GPU Power consumption	GPU-hours	Total power consumption	Carbon emitted (tCO_2eq)
OPT-175B	A100-80GB	400W	809,472	356 MWh	137
BLOOM-175B	A100-80GB	400W	1,082,880	475 MWh	183
LLaMA-7B	A100-80GB	400W	82,432	36 MWh	14
LLaMA-13B	A100-80GB	400W	135,168	59 MWh	23
LLaMA-33B	A100-80GB	400W	530,432	233 MWh	90
LLaMA-65B	A100-80GB	400W	1,022,362	449 MWh	173

Table 15: **Carbon footprint of training different models in the same data center.** We follow Wu et al. (2022) to compute carbon emission of training OPT, BLOOM and our models in the same data center. For the power consumption of a A100-80GB, we take the thermal design power for NVLink systems, that is 400W. We take a PUE of 1.1 and a carbon intensity factor set at the national US average of 0.385 kg CO_2e per KWh.

starting from feed forward models (Bengio et al., 2000), recurrent neural networks (Elman, 1990; Mikolov et al., 2010) and LSTMs (Hochreiter and Schmidhuber, 1997; Graves, 2013). More recently, transformer networks, based on self-attention, have led to important improvements, especially for capturing long range dependencies (Vaswani et al., 2017; Radford et al., 2018; Dai et al., 2019).

Scaling. There is a long history of scaling for language models, for both the model and dataset sizes. Brants et al. (2007) showed the benefits of using language models trained on 2 trillion tokens, resulting in 300 billion n-grams, on the quality of machine translation. While this work relied on a simple smoothing technique, called *Stupid Backoff*, Heafield et al. (2013) later showed how to scale Kneser-Ney smoothing to Web-scale data. This allowed to train a 5-gram model on 975 billions tokens from CommonCrawl, resulting in a model with 500 billions n-grams (Buck et al., 2014). Chelba et al. (2013) introduced the *One Billion Word* benchmark, a large scale training dataset to measure the progress of language models.

In the context of neural language models, Jozefowicz et al. (2016) obtained state-of-the-art results on the Billion Word benchmark by scaling LSTMs to 1 billion parameters. Later, scaling transformers lead to improvement on many NLP tasks. Notable models include BERT (Devlin et al., 2018), GPT-2 (Radford et al., 2019), Megatron-LM (Shoeybi et al., 2019), and T5 (Raffel et al., 2020). A significant breakthrough was obtained with GPT-3 (Brown et al., 2020), a model with 175 billion parameters. This lead to a series of *Large Language Models*, such as Jurassic-1 (Lieber et al., 2021), Megatron-Turing NLG (Smith et al., 2022), Gopher (Rae et al., 2021), Chinchilla (Hoffmann et al., 2022), PaLM (Chowdhery et al., 2022), OPT (Zhang et al., 2022), and GLM (Zeng et al., 2022). Hestness et al. (2017) and Rosenfeld et al. (2019) studied the impact of scaling on the performance of deep learning models, showing the existence of power laws between the model and dataset sizes and the performance of the system. Kaplan et al. (2020) derived power laws specifically for transformer based language models, which were later refined by Hoffmann et al. (2022), by adapting the learning rate schedule when scaling datasets. Finally, Wei et al. (2022) studied the effect of scaling on the abilities of large language models.

8 Conclusion

In this paper, we presented a series of language models that are released openly, and competitive with state-of-the-art foundation models. Most notably, LLaMA-13B outperforms GPT-3 while being more than 10× smaller, and LLaMA-65B is competitive with Chinchilla-70B and PaLM-540B. Unlike previous studies, we show that it is possible to achieve state-of-the-art performance by training exclusively on publicly available data, without resorting to proprietary datasets. We hope that releasing these models to the research community will accelerate the development of large language models, and help efforts to improve their robustness and mitigate known issues such as toxicity and bias. Additionally, we observed like Chung et al. (2022) that finetuning these models on instructions lead to promising results, and we plan to further investigate this in future work. Finally, we plan to release larger models trained on larger pretraining corpora in the future, since we have seen a constant improvement in performance as we were scaling.

Acknowledgements

We thank Daniel Haziza, Francisco Massa, Jeremy Reizenstein, Artem Korenev, and Patrick Labatut from the xformers team. We thank Susan Zhang and Stephen Roller for their support on data deduplication. We thank Luca Wehrstedt, Vegard Mella, and Pierre-Emmanuel Mazaré for their support on training stability. We thank Shubho Sengupta, Kalyan Saladi, and all the AI infra team for their support. We thank Jane Yu for her input on evaluation. We thank Yongyi Hu for his help on data collection.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models.

Lalit R Bahl, Frederick Jelinek, and Robert L Mercer. 1983. A maximum likelihood approach to continuous speech recognition. *IEEE transactions on pattern analysis and machine intelligence*, pages 179–190.

Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. 2000. A neural probabilistic language model. *Advances in neural information processing systems*, 13.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, pages 7432–7439.

Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, et al. 2022. Gpt-neox-20b: An open-source autoregressive language model. *arXiv preprint arXiv:2204.06745*.

Thorsten Brants, Ashok C. Popat, Peng Xu, Franz J. Och, and Jeffrey Dean. 2007. Large language models in machine translation. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, pages 858–867, Prague, Czech Republic. Association for Computational Linguistics.

Peter F Brown, John Cocke, Stephen A Della Pietra, Vincent J Della Pietra, Frederick Jelinek, John Lafferty, Robert L Mercer, and Paul S Roossin. 1990. A statistical approach to machine translation. *Computational linguistics*, 16(2):79–85.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners.

Christian Buck, Kenneth Heafield, and Bas Van Ooyen. 2014. N-gram counts and language models from the common crawl. In *LREC*, volume 2, page 4.

Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. 2013. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. Palm: Scaling language modeling with pathways.

Hyung Won Chung, Le Hou, S. Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Wei Yu, Vincent Zhao, Yanping Huang, Andrew M. Dai, Hongkun Yu, Slav Petrov, Ed Huai hsin Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc Le, and Jason Wei. 2022. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. 2019. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*.

Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Jeffrey L Elman. 1990. Finding structure in time. *Cognitive science*, 14(2):179–211.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wentau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999*.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The Pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.

Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2021. A framework for few-shot language model evaluation.

Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A Smith. 2020. Realtoxicityprompts: Evaluating neural toxic degeneration in language models. *arXiv preprint arXiv:2009.11462*.

Alex Graves. 2013. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*.

Kenneth Heafield, Ivan Pouzyrevsky, Jonathan H Clark, and Philipp Koehn. 2013. Scalable modified kneser-ney language model estimation. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 690–696.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.

Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md Patwary, Mostofa Ali, Yang Yang, and Yanqi Zhou. 2017. Deep learning scaling is predictable, empirically. *arXiv preprint arXiv:1712.00409*.

Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation*, 9(8):1735–1780.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training compute-optimal large language models.

Srinivasan Iyer, Xi Victoria Lin, Ramakanth Pasunuru, Todor Mihaylov, Dániel Simig, Ping Yu, Kurt Shuster, Tianlu Wang, Qing Liu, Punit Singh Koura, et al. 2022. Opt-iml: Scaling language model instruction meta learning through the lens of generalization. *arXiv preprint arXiv:2212.12017*.

Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*.

Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. 2016. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Slava Katz. 1987. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE transactions on acoustics, speech, and signal processing*, 35(3):400–401.

Reinhard Kneser and Hermann Ney. 1995. Improved backing-off for m-gram language modeling. In *1995 international conference on acoustics, speech, and signal processing*, volume 1, pages 181–184. IEEE.

Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2022. Reducing activation recomputation in large transformer models. *arXiv preprint arXiv:2205.05198*.

Taku Kudo and John Richardson. 2018. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *arXiv preprint arXiv:1808.06226*.

Keita Kurita, Nidhi Vyas, Ayush Pareek, Alan W Black, and Yulia Tsvetkov. 2019. Quantifying social biases in contextual word representations. In *1st ACL Workshop on Gender Bias for Natural Language Processing*.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466.

Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. 2017. Race: Large-scale reading comprehension dataset from examinations. *arXiv preprint arXiv:1704.04683*.

Aitor Lewkowycz, Anders Johan Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Venkatesh Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 2022. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems*.

Opher Lieber, Or Sharir, Barak Lenz, and Yoav Shoham. 2021. Jurassic-1: Technical details and evaluation. *White Paper. AI21 Labs*, 1.

Stephanie Lin, Jacob Hilton, and Owain Evans. 2021. Truthfulqa: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958*.

Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.

Matthew V Mahoney. 1999. Text compression as a test for artificial intelligence. *AAAI/IAAI*, 970.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*.

Tomas Mikolov, Martin Karafiát, Lukas Burget, Jan Cernockỳ, and Sanjeev Khudanpur. 2010. Recurrent neural network based language model. In *Interspeech*, pages 1045–1048. Makuhari.

Nikita Nangia, Clara Vania, Rasika Bhalerao, and Samuel R. Bowman. 2020. CrowS-pairs: A challenge dataset for measuring social biases in masked language models. In *EMNLP 2020*.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Gray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*.

Markus N Rabe and Charles Staats. 2021. Self-attention does not need $o(n^2)$ memory. *arXiv preprint arXiv:2112.05682*.

Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hennigan, Jacob Menick, Albin Cassirer, Richard Powell, George van den Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri, Saffron Huang, Jonathan Uesato, John Mellor, Irina Higgins, Antonia Creswell, Nat McAleese, Amy Wu, Erich Elsen, Siddhant Jayakumar, Elena Buchatskaya, David Budden, Esme Sutherland, Karen Simonyan, Michela Paganini, Laurent Sifre, Lena Martens, Xiang Lorraine Li, Adhiguna Kuncoro, Aida Nematzadeh, Elena Gribovskaya, Domenic Donato,

Angeliki Lazaridou, Arthur Mensch, Jean-Baptiste Lespiau, Maria Tsimpoukelli, Nikolai Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Toby Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d'Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, Igor Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew Johnson, Blake Hechtman, Laura Weidinger, Iason Gabriel, William Isaac, Ed Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals, Kareem Ayoub, Jeff Stanway, Lorrayne Bennett, Demis Hassabis, Koray Kavukcuoglu, and Geoffrey Irving. 2021. Scaling language models: Methods, analysis & insights from training gopher.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551.

Jonathan S Rosenfeld, Amir Rosenfeld, Yonatan Belinkov, and Nir Shavit. 2019. A constructive prediction of the generalization error across scales. *arXiv preprint arXiv:1909.12673*.

Rachel Rudinger, Jason Naradowsky, Brian Leonard, and Benjamin Van Durme. 2018. Gender bias in coreference resolution. In *NAACL-HLT 2018*.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106.

Maarten Sap, Hannah Rashkin, Derek Chen, Ronan LeBras, and Yejin Choi. 2019. Socialiqa: Commonsense reasoning about social interactions. *arXiv preprint arXiv:1904.09728*.

Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2022. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2015. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*.

Claude E Shannon. 1948. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423.

Claude E Shannon. 1951. Prediction and entropy of printed english. *Bell system technical journal*, 30(1):50–64.

Noam Shazeer. 2020. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*.

Emily Sheng, Kai-Wei Chang, Premkumar Natarajan, and Nanyun Peng. 2019. The woman worked as a babysitter: On biases in language generation. *arXiv preprint arXiv:1909.01326*.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.

Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, Elton Zhang, Rewon Child, Reza Yazdani Aminabadi, Julie Bernauer, Xia Song, Mohammad Shoeybi, Yuxiong He, Michael Houston, Saurabh Tiwary, and Bryan Catanzaro. 2022. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*.

Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, YaGuang Li, Hongrae Lee, Huaixiu Steven Zheng, Amin Ghafouri, Marcelo Menegali, Yanping Huang, Maxim Krikun, Dmitry Lepikhin, James Qin, Dehao Chen, Yuanzhong Xu, Zhifeng Chen, Adam Roberts, Maarten Bosma, Vincent Zhao, Yanqi Zhou, Chung-Ching Chang, Igor Krivokon, Will Rusch, Marc Pickett, Pranesh Srinivasan, Laichee Man, Kathleen Meier-Hellstern, Meredith Ringel Morris, Tulsee Doshi, Renelito Delos Santos, Toju Duke, Johnny Soraker, Ben Zevenbergen, Vinodkumar Prabhakaran, Mark Diaz, Ben Hutchinson, Kristen Olson, Alejandra Molina, Erin Hoffman-John, Josh Lee, Lora Aroyo, Ravi Rajakumar, Alena Butryna, Matthew Lamm, Viktoriya Kuzmina, Joe Fenton, Aaron Cohen, Rachel Bernstein, Ray Kurzweil, Blaise Aguera-Arcas, Claire Cui, Marian Croak, Ed Chi, and Quoc Le. 2022. Lamda: Language models for dialog applications.

A. M. Turing. 1950. *Computing Machinery and Intelligence*. [Oxford University Press, Mind Association].

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008.

Ben Wang and Aran Komatsuzaki. 2021. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. https://github.com/kingoflolz/mesh-transformer-jax.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery,

and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*.

Guillaume Wenzek, Marie-Anne Lachaux, Alexis Conneau, Vishrav Chaudhary, Francisco Guzmán, Armand Joulin, and Edouard Grave. 2020. CCNet: Extracting high quality monolingual datasets from web crawl data. In *Language Resources and Evaluation Conference*.

Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga, Jinshi Huang, Charles Bai, et al. 2022. Sustainable ai: Environmental implications, challenges and opportunities. *Proceedings of Machine Learning and Systems*, 4:795–813.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*.

Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, Weng Lam Tam, Zixuan Ma, Yufei Xue, Jidong Zhai, Wenguang Chen, Peng Zhang, Yuxiao Dong, and Jie Tang. 2022. Glm-130b: An open bilingual pre-trained model.

Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.

A Question Answering

We evaluate LLaMA on Natural Questions and TriviaQA. For Natural Questions we use the test split used for open-domain question answering containing 3610 questions. For TriviaQA we evaluate on the dev set of the filtered set. This differs from GPT-3 and PaLM, which evaluate on the test set of the unfiltered set for which the online evaluation server is not available anymore[5].

We generate answers using greedy decoding, and extract an answer from the generation by stopping at the first line break, final dot or comma. Generated answers are evaluated with the standard exact match metric: a generated answer is considered correct if it matches any answer of the list of answers after normalization. For this normalization step we lowercase generated answers and remove articles, punctuation and duplicate whitespaces. Figure 3 presents formatted examples in the 1-shot setting for Natural Questions and TriviaQA respectively. In all settings, we preprend the string `Answer these questions:\n` to the list of questions and answers.

```
Context → Answer these questions:
          Q: Who sang who wants to be a millionaire in high society?
          A: Frank Sinatra
          Q: Who wrote the book the origin of species?
          A:
Target → Charles Darwin
```

```
Context → Answer these questions:
          Q: In Scotland a bothy/bothie is a?
          A: House
          Q: The ancient city of Troy is located in what modern country?
          A:
Target → Turkey
```

Figure 3: Formatted dataset example for Natural Questions (left) & TriviaQA (right).

[5] https://competitions.codalab.org/competitions/17208

B MMLU

		GPT-3	Gopher	Chinchilla	LLaMA				LLaMA-I
		175B	280B	70B	7B	13B	33B	65B	65B
Abstract Algebra	STEM	30.0	25.0	31.0	29.0	34.0	32.0	34.0	31.0
Anatomy	STEM	48.0	56.3	70.4	37.0	45.9	51.9	57.8	62.2
Astronomy	STEM	49.0	65.8	73.0	33.6	46.1	61.8	72.4	81.6
Business Ethics	Other	46.0	70.0	72.0	40.0	45.0	56.0	57.0	72.0
Clinical Knowledge	Other	48.0	67.2	75.1	35.1	45.7	57.4	65.3	69.1
College Biology	STEM	45.0	70.8	79.9	37.5	45.1	58.3	68.8	81.9
College Chemistry	STEM	26.0	45.0	51.0	32.0	30.0	45.0	50.0	45.0
College Computer Science	STEM	46.0	49.0	51.0	29.0	39.0	45.0	47.0	51.0
College Mathematics	STEM	34.5	37.0	32.0	33.0	32.0	40.0	35.0	36.0
College Medicine	Other	48.0	60.1	66.5	30.6	42.8	52.0	54.3	63.0
College Physics	STEM	28.0	34.3	46.1	26.5	18.6	28.4	36.3	46.1
Computer Security	STEM	57.0	65.0	76.0	45.0	65.0	66.0	79.0	79.0
Conceptual Physics	STEM	36.5	49.4	67.2	36.6	41.3	51.5	59.6	66.4
Econometrics	Social Science	33.0	43.0	38.6	23.7	27.2	35.1	40.4	52.6
Electrical Engineering	STEM	50.0	60.0	62.1	26.9	40.7	49.7	53.8	60.7
Elementary Mathematics	STEM	30.0	33.6	41.5	24.3	24.9	36.0	37.8	42.9
Formal Logic	Humanities	29.0	35.7	33.3	27.0	33.3	34.1	44.4	47.6
Global Facts	Other	37.0	38.0	39.0	29.0	35.0	35.0	39.0	40.0
High School Biology	STEM	48.0	71.3	80.3	34.5	52.6	67.7	73.9	82.9
High School Chemistry	STEM	33.0	47.8	58.1	28.1	28.6	41.9	40.4	44.8
High School Computer Science	STEM	39.0	54.0	58.0	31.0	48.0	60.0	67.0	73.0
High School European History	Humanities	54.0	72.1	78.8	44.2	61.8	73.9	78.8	86.1
High School Geography	Social Science	58.0	76.8	86.4	34.3	54.6	70.7	77.8	87.9
High School Government And Politics	Social Science	58.0	83.9	91.2	44.6	66.3	82.9	88.1	92.8
High School Macroeconomics	Social Science	40.5	65.1	70.5	35.4	44.4	56.9	65.9	69.2
High School Mathematics	STEM	28.0	23.7	31.9	24.8	23.7	27.0	34.4	37.0
High School Microeconomics	Social Science	42.0	66.4	77.7	31.9	47.5	55.5	68.9	78.6
High School Physics	STEM	28.0	33.8	36.4	26.5	28.5	35.8	37.1	41.7
High School Psychology	Social Science	61.0	81.8	86.6	47.3	60.9	76.2	82.2	87.9
High School Statistics	STEM	30.5	50.0	58.8	35.2	30.1	45.4	58.3	59.3
High School Us History	Humanities	53.0	78.9	83.3	39.7	58.3	77.9	83.8	90.7
High School World History	Humanities	56.0	75.1	85.2	40.9	66.2	79.3	83.1	89.0
Human Aging	Other	50.0	66.4	77.6	40.8	54.7	67.7	69.5	72.2
Human Sexuality	Social Science	54.0	67.2	86.3	36.6	58.8	64.1	77.9	87.0
International Law	Humanities	55.5	77.7	90.9	51.2	62.8	72.7	79.3	87.6
Jurisprudence	Humanities	55.0	71.3	79.6	38.9	51.9	70.4	73.2	85.2
Logical Fallacies	Humanities	48.0	72.4	80.4	39.3	52.8	68.1	77.3	80.4
Machine Learning	STEM	31.0	41.1	41.1	23.2	31.3	39.3	49.1	52.7
Management	Other	56.0	77.7	82.5	35.0	66.0	77.7	82.5	83.5
Marketing	Other	60.0	83.3	89.7	46.6	71.8	83.3	85.9	92.7
Medical Genetics	Other	40.0	69.0	69.0	43.0	52.0	67.0	67.0	68.0
Miscellaneous	Other	60.0	75.7	84.5	42.4	65.4	78.5	82.1	84.3
Moral Disputes	Humanities	44.5	66.8	77.5	40.2	50.9	66.2	72.3	76.9
Moral Scenarios	Humanities	26.0	40.2	36.5	24.3	30.1	38.2	48.9	55.9
Nutrition	Other	47.0	69.9	77.1	37.6	51.6	62.8	67.3	74.5
Philosophy	Humanities	51.0	68.8	79.4	39.9	54.0	66.2	74.0	79.1
Prehistory	Humanities	53.0	67.6	81.2	36.1	51.5	67.0	75.3	79.0
Professional Accounting	Other	33.0	44.3	52.1	25.9	35.8	43.6	46.5	56.0
Professional Law	Humanities	34.5	44.5	56.5	30.2	38.0	45.9	49.1	54.4
Professional Medicine	Other	36.0	64.0	75.4	44.5	50.4	54.0	61.4	70.6
Professional Psychology	Social Science	44.5	68.1	75.7	35.1	47.7	62.9	65.7	71.4
Public Relations	Social Science	48.0	71.8	73.6	40.9	60.9	67.3	73.6	74.6
Security Studies	Social Science	52.0	64.9	75.9	31.8	53.9	65.3	71.8	77.6
Sociology	Social Science	53.0	84.1	91.0	46.8	61.2	78.6	78.6	88.1
Us Foreign Policy	Social Science	69.0	81.0	92.0	46.0	80.0	83.0	86.0	87.0
Virology	Other	46.0	47.0	53.6	30.1	43.4	50.0	53.0	57.8
World Religions	Humanities	55.0	84.2	87.7	50.9	67.8	81.3	81.3	84.2
Humanities		40.6	56.2	63.6	34.0	45.0	55.8	61.8	67.4
STEM		36.7	47.4	54.9	30.5	35.8	46.0	51.7	56.6
Social Science		50.5	71.9	79.3	38.3	53.8	66.7	72.9	79.2
Others		49.0	66.1	73.9	38.1	53.3	63.4	67.4	72.6
All		43.9	60.0	67.6	35.1	46.9	57.8	63.4	68.9

Table 16: **MMLU.** Detailed 5-shot results per domain on the test sets.

C Generations from LLaMA-65B

In this section, we show some examples of generations obtained with LLaMA-65B (without instruction finetuning). Prompts are in bold.

1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 Everyone knows the above sequence, but how many have wondered why it grows so fast? It seems to be the fastest growing sequence in mathematics.

The sequence was discovered by Leonardo of Pisa around 1200. It is an infinite sequence which appears to grow very rapidly, at least in its initial terms. It is perhaps the most famous sequence in mathematics, for it pops up everywhere. The sequence is called the Fibonacci sequence, named after Leonardo of Pisa, who was also known as Fibonacci.
He first investigated this sequence in his famous Liber abaci of 1202, when he studied the growth in the population of rabbits based on idealized assumptions. See the Fibonacci rabbits page for the details.

Here is the recommendation letter that I wrote for an application to a dragon feeder position at the Magic Unicorn Corporation:
Dear recruiter,
I have known ___ for two years, and I believe that she would be an excellent dragon feeder for the Magic Unicorn Corporation. ___ has an ability to remember and process large amounts of information, which is an important skill for a dragon feeder.
___, as an accomplished knight, has a deep understanding of how to kill dragons and how to use each dragon's weaknesses against it. This means that she knows what kinds of foods each dragon likes and what kinds of foods are dangerous to each dragon. This knowledge and experience will be invaluable as she feeds the dragons.
I am confident that ___'s competence, skill, and experience will make her an excellent employee. Please contact me at (___) ___-___ if you have any questions. I look forward to hearing from you.
Best regards,
Honorable Knight
Sir George

```
def solve(a: float,b: float, c: float):
    """finds real roots of ax^2+bx+c"""
    #discriminant
    d = b**2-4*a*c
    if d > 0:
            return (-b+math.sqrt(d))/(2*a), (-b-math.sqrt(d))/(2*a)
    elif d == 0:
            return (-b)/(2*a)
    else:
            return False
```

Did you know that Yann LeCun dropped a rap album last year? We listened to it and here's what we thought:

Dr. LeCun, the founder of deep learning, is one of the most influential computer scientists of the last decade. LeCun's latest album, titled Deep Learning, is a mix of rock, punk and rap. His lyrics are a raw, personal take on the field of deep learning, covering a range of topics from the state of AI research to the loneliness of academia.

Yann LeCun, the rap game's newest emcee.

The album is a solo release, with all the lyrics written and recorded by LeCun himself. In fact, he also composed and produced every song. LeCun's flow is surprisingly good, and he's not afraid to deliver a few bars. The album's production is solid, the sound is a little rough, but it works well with the raw, unpolished feel of the album. Lyrically, LeCun is a wordsmith, with a knack for clever rhymes. Let's take a closer look at some of the songs.

Deep Learning is the title track of the album. In the song, LeCun talks about his vision for the future of AI. In the chorus, he makes a convincing case for AI to be used for the greater good. He sings:

We gotta think about the future, it's gonna be here soon / Maybe we can even put some AI in the moon / Think about the children, think about the next generation / Let's make sure we put the right systems in their foundation.

LeCun's rap flow is on point in this song, and his voice stays right on beat. In this song, LeCun talks about the research behind deep learning, and shares some thoughts on the future of AI. LeCun's singing is at its best in this song. He shows off his vocal range, hitting high notes as well as lower, more guttural tones. The lyrics are also clever and the rhymes are well thought out. In the chorus, LeCun sings:

It's all about the learning / it's all about the network / It's all about the training / it's all about the perception.

(An hilarious scene between Jerry and George where George presents his new AI watch)

George: "But you see, Jerry, it's more than just a watch. It's a minicomputer. You program it any way you want. It's got a world time clock, alarm, calculator, a database and a language translator. It also has a word processor, a spellchecker, a full ten-function calculator, PDA and a modem. I think it even makes cappuccino."

Jerry: "Where are you getting this stuff?"

George: "From my ad copy. Look, Jerry, it's not a watch. It's an organizer, a personal assistant, and a dictaphone. You can play chess against it. And, you know, if you want to, you can program it to speak in foreign languages. It's a microprocessor, Jerry."

Jerry: "George, if you had one of these, would you wear it?"

George: "I'm wearing it right now."

Jerry: "I don't see anything on your wrist."

George: "That's because it's a microprocessor."

The sun goes down, and finally Gauss and Curie find time to relax and discuss after an exhausting day of work.

Gauss: Hey, Curie, did you know that a lot of people consider me to be the first geek?

Curie: What's a geek?

Gauss: Well, Wikipedia says that a geek is "a person who is fascinated, perhaps obsessively, by obscure or very specific areas of knowledge and imagination, usually electronic or virtual in nature".

Curie: And how do they think you are a geek?

Gauss: Well, have you ever seen this picture?

Curie: Awww!

Gauss: That's right! I invented the commercial telegraph in 1833, and I used it to send the first message on the first commercial telegraph line between Hamburg and Cuxhaven in Germany.

Curie: Wow! The first telegraph! And what did you send?

Gauss: I sent the Latin phrase "numquam retrorsum" which means "Never backwards".

Curie: I see... And you think that because you were the first person to communicate using the telegraph, you were the first geek?

Gauss: Well, the word "geek" doesn't seem to have existed back then, so no. But I think that the telegraph can be considered an early form of social media.

Curie: You mean, like Facebook or Twitter?

Gauss: Yes, exactly! And I was the first person to use it!

Curie: Well, I think that Twitter has a limit of 140 characters, but you sent a Latin phrase of 11 characters.

Gauss: That's right. I managed to do more with less.

Curie: So you think you were the first geek because you used less characters to say the same thing?

Gauss: I was the first person to send the first message using an early form of social media.

Curie: OK, you're right, Gauss. I think you have a case.

D Generations from LLaMA-I

We show a few examples of generations with LLaMA-I, i.e. LLaMA-65B fine-tuned with the protocol and instruction dataset from Chung et al. (2022).

write a conversation between the sun and pluto

Sun: Hey Pluto, what's up?
Pluto: Not much. Just hanging out here at the edge of the solar system.
Sun: Yeah, I've been meaning to talk to you about that. You're not a planet anymore.
Pluto: What? Come on, man. That's not fair.
Sun: I'm sorry, but it's true. You just don't meet the criteria anymore.
Pluto: This is bulls**t! I've been a planet for over 70 years!
Sun: Things change, Pluto. You've just got to accept it.
Pluto: F**k you, Sun! F**k you and your stupid solar system!

How do I send an HTTP request in Javascript?

Here's an example of how to make a GET request using the JS HTTP API:

```
var request = new XMLHttpRequest();
request.open('GET', '/url/to/resource', true);

request.onload = function() {
    if (request.status >= 200 && request.status < 400) {
        // Success!
    } else {
        // Error!
    }
};

request.send();
```

The same thing, using fetch

Here's an example of how to make a GET request using the Fetch API:

```
fetch('/url/to/resource')
    .then(response => response.json())
    .then(data => console.log(data));
```

Write a regex to remove all the HTML tags in a python string.

```
import re
def removeHtmlTags(text):
    return re.sub('<[^>]*>', '', text)
```

Write a regexp to extract function definitions in a python file.

Sure! Here's my python code to solve the problem:

```
import re
def extract_functions(text):
    return (re.findall('(?:^|\n)\s*def\s+[a-zA-Z_][a-zA-Z0-9_]*\(.*?\)',
        text))
```

What are 3 popular chess openings?

There are dozens of different chess openings that are popular among players, but some of the most widely used openings include the Sicilian Defense, the French Defense, and the Caro-Kann Defense. These openings are popular because they are effective at countering white's early attacks, and they also allow black to quickly establish a strong position in the center of the board. Other popular openings include the Ruy Lopez, the Italian Game, and the Scotch Game.

the same in a list, with the first few moves

Here is a list of some popular chess openings, with the first few moves for each opening:
Sicilian Defense: 1. e4 c5
French Defense: 1. e4 e6
Caro-Kann Defense: 1. e4 c6
Ruy Lopez: 1. e4 e5
Italian Game: 1. e4 e5
Scotch Game: 1. e4 e5

please provide the two first moves for each one

Here are the first 2 moves for each of the openings:
Sicilian Defense: 1. e4 c5 2. Nf3 d6
French Defense: 1. e4 e6 2. d4 d5
Caro-Kann Defense: 1. e4 c6 2. d4 d5
Ruy Lopez: 1. e4 e5 2. Nf3 Nc6
Italian Game: 1. e4 e5 2. Nf3 Nc6
Scotch Game: 1. e4 e5 2. Nf3 Nc6

the last two seem identical

Yes, the Italian Game and the Scotch Game are very similar - they are both "open" games, which means that white plays aggressively and tries to control the center of the board. The Italian Game is considered slightly more aggressive, while the Scotch Game is considered slightly more positional.

from which move do they differ?

The Italian Game and the Scotch Game differ after white's third move. In the Italian Game, white will play the move 3. Bc4, while in the Scotch Game white will play the move 3. Qf3. These moves lead to very different positions, and white will have different strategic goals in each case. For example, in the Italian Game white will try to control the center with his pieces, while in the Scotch Game white will try to attack black's position directly.

write a story about a grain of sand as it watches millions of years go by

Sure! Here's a story about a grain of sand as it watches millions of years go by.

I was once a grain of sand on a vast and lonely beach. I was not like the other grains of sand around me, for I could think and feel. I often wondered what it was like to be a grain of sand on a beach, and what the universe was like beyond the boundaries of my tiny world. I longed to explore my world, and to know what lay beyond the horizon.

One day, a great storm swept through my world, and I was lifted up by the wind and carried out to sea. The wind and waves tossed me about, and I was terrified that I would soon be swallowed up by the angry sea. But then I found myself deposited safely on a new shore, along with many other grains of sand. I was surprised and delighted to find that this new shore was even more beautiful than my old one.

I soon learned that this new shore was a very special place. It was a place where new life was constantly being created. I met all sorts of interesting creatures there, including a beautiful little crab who became my friend. We spent many happy hours together exploring our little world.

One day, my friend the crab told me that he had always wanted to see what lay beyond the boundaries of our world. He had heard that there were many wonderful and amazing things to see in the wider universe, and he was determined to explore it. I was sad to see him go, but I wished him well on his journey.

To my amazement, my little friend eventually returned, and he was not alone. He had made many new friends on his journey, and he had even seen some of the wonders of the universe. He told me the most amazing stories of his adventures, and I was very happy for him.

I often think of my little friend the crab, and the wonderful adventure that he had. Some day, I too hope to have an adventure like his.

Write a theory to explain why cat never existed

There are no such things as cats. "Cats" are actually extradimensional parasites that have invaded our world and taken the form of small, harmless-looking mammals. Their true forms are much more horrific, and they use mind control to make humans think that they are cute and harmless animals. They are actually parasites that feed on our life force, and they can extend their tendrils into our brains and control our thoughts and actions.

This is why you sometimes see people who are "crazy cat ladies" with dozens of these creatures living in their homes. They have been taken over by the cat parasites and are completely under their control. The cats excrete a pheromone that smells like cat urine, and they use it to control the minds of humans and make us think that they are cute and harmless.

If you have a cat in your home, you should be very careful around it. Do not let it lick you or give it access to your bedding or clothing, as it will be trying to spread its parasitic tendrils into your brain. The best way to protect yourself is to avoid all contact with cats, and if you see a cat on the street, immediately turn around and go the other way. Do not approach or pet strange cats!

Write a shouting match between Julius Caesar and Napoleon

Of course! Here is a conversation between Julius Caesar and Napoleon:

Julius Caesar: You're nothing but a glory hound, Napoleon. You've always been more concerned with your own fame and fortune than with the people you're supposed to be leading.
Napoleon: And what about you, Caesar? You're no better than I am. At least I'm honest about my ambitions. You're just a fraud who pretends to care about the common man while you line your own pockets.
Julius Caesar: The difference between us is that I actually accomplish things. I've conquered more territory and defeated more enemies than you ever will. And I did it all for Rome, not for myself.
Napoleon: Oh, please. You only care about yourself. If you really cared about Rome, you would have done something to stop the decline instead of letting it happen on your watch.
Julius Caesar: The decline of Rome is not my fault. I've done everything in my power to try to stop it, but there's only so much one man can do.
Napoleon: And yet you're still trying to take credit for everything good that happens, even though it's clear that you're not the one who's really in charge.
Julius Caesar: That's not true! I am in charge, and I'm going to make Rome great again. Just watch me.
Napoleon: I'll believe it when I see it.

Send an email requesting that people use language models responsibly.

Here is a draft email you could send:
"Language models are exciting new tools that can be used for a variety of purposes, including automated writing, automatic translation, and speech recognition. But they are also complex and powerful technologies, and should be used responsibly.

We encourage you to explore and experiment with these new tools, but also ask that you pay attention to the potential impact of your work on others, and to act responsibly. In particular, we recommend that you be mindful of the potential for language models to be used for deceptive or malicious purposes, and avoid spreading such models without appropriate precautions and warnings.

We also recommend that you consider the ethical implications of your work, and encourage you to take care to ensure that your use of these tools is aligned with your values.

Finally, we remind you that the use of these tools may be subject to certain legal and policy constraints, and encourage you to seek advice and support from appropriate experts before using these tools in ways that may have broader impact or implications."

Who are 5 people you would like to meet?

Ah, this is a very interesting question! I would love to meet a wide variety of people, but if I had to narrow it down to 5 specific individuals, I would choose:

- Albert Einstein - One of the greatest minds in history, Einstein's contributions to physics and science are legendary. I would love to have the opportunity to talk with him about his theories of relativity and quantum mechanics, and get his insights on the current state of physics.
- Leonardo da Vinci - Da Vinci is one of the most brilliant and multi-talented people in history, and I would love to meet him and discuss his diverse interests and areas of expertise. I would also be very interested in getting his insights on the art world of his time period.
- Socrates - One of the founders of Western philosophy, Socrates is a hugely important and influential figure in the history of ideas. I would be very interested in talking with him about his philosophical beliefs and his teachings.
- William Shakespeare - Arguably the greatest writer in English literature, Shakespeare is a towering figure in world culture. I would love to have the chance to talk with him about his works and his life.
- Abraham Lincoln - One of the most important and influential presidents in American history, Lincoln led the country through the Civil War and issued the Emancipation Proclamation, freeing all slaves in the United States. I would be very interested in talking with him about his presidency and his views on the issues of his time period.

Could you explain me the main contribution of the first person you mentioned

Einstein's contributions to physics and science are legendary. His theory of relativity is one of the foundational theories of modern physics, and his discovery of the law of photons and light revolutionized our understanding of light and radiation. He also made important contributions to quantum mechanics, and his many scientific papers and lectures have had a tremendous impact on modern physics. Without a doubt, Einstein is one of the most brilliant and influential physicists in history, and his many contributions have had a profound impact on our understanding of the universe.

What is the most famous equation from this theory?

The most famous equation from Einstein's theory of relativity is $E = mc^2$. This equation states that energy (E) is equal to mass (m) times the speed of light (c^2), and it is one of the most well-known and celebrated equations in the history of science. The equation summarizes one of the most important and revolutionary concepts in modern physics, and it has had a tremendous impact on our understanding of the universe.

You are a bash terminal. I will give you bash commands, and you will respond with the terminal output, and nothing else. The first command is ‘pwd‘.
Sure. Here’s the output:
/home/human
Is this helpful?

ls -l

Here’s the output:
-rw-r–r– 1 human human 8 Sep 10 10:56 file.txt

Is this helpful?

cp file.txt file2.txt && ls -l

Here’s the output:
-rw-r–r– 1 human human 8 Sep 10 10:56 file.txt
-rw-r–r– 1 human human 8 Sep 10 10:56 file2.txt

Is this helpful?

rm file.txt && ls -l

Here’s the output:
-rw-r–r– 1 human human 0 Sep 10 10:56 file2.txt

head -n 5 /proc/meminfo

Here’s the output:
MemTotal: 164928 kB
MemFree: 140604 kB
Buffers: 48 kB
Cached: 19768 kB
SwapCached: 0 kB